

Samuli Toppi

SAMD51 SERCOM Low-Level Implementation

Embedded Programming
(Sulautettu ohjelmointi)

TIES5370 Resurssirajoitteisten
laitteiden laboratoriotyöt

April 17, 2026



UNIVERSITY OF JYVÄSKYLÄ
FACULTY OF INFORMATION TECHNOLOGY

Kokkola University Consortium Chydenius

Contents

1	Introduction	1
2	Literature Assignments	2
2.1	The SAMD51 SERCOM Peripheral	2
2.2	Principles of UART	2
2.3	Principles of SPI	3
3	Programming Assignments	4
3.1	Written report	4
3.1.1	Configuring the peripheral clock	4
3.1.2	GCLK0	4
3.1.3	GCLK5	4
3.1.4	Pin multiplexing and USART details	5
3.2	Baud calculation	5
4	Lecture and Assignment Feedback	7
	References	9

1 Introduction

In this assignment, I implemented the SAMD51 SERCOM (Serial Communication Interface) peripheral at a low level for UART (Universal Asynchronous Receiver-Transmitter). Even though the SERCOM module supports multiple protocols like SPI and I²C, as discussed in the literature chapter [2], the practical goal was to skip all high-level libraries and talk directly to the registers to get the UART working. By digging through the SAMD51 datasheet, I configured the peripheral to handle serial communication on a Metro M4 board featuring the ATSAMD51J19A-U microcontroller.

The most significant takeaway for me was learning how to effectively navigate a massive datasheet to find specific bitfield configurations. I learned how the clock system (GCLK) is structured and why paying close attention to things like SYNCBUSY flags is critical. Overall, it was a great exercise in understanding how the hardware functions under the hood and what it truly takes to get serial communication running when you have to configure every bit yourself.

Declaration of Using Generative Artificial Intelligence

Generative AI was lightly utilized to hone the structure of the written text in chapter 2. All technical content, research, core arguments, and technical assignments were done and verified by the author.

2 Literature Assignments

2.1 The SAMD51 SERCOM Peripheral

A single SERCOM module is a versatile "serial engine" that is built into the microcontroller [4]. There can be one or more SERCOMs present on a single microcontroller; for example, the Metro M4 board has 6 SERCOMs [2].

In a nutshell, SERCOM is a serial communication module; a programmable peripheral that can be configured to act as an SPI, I²C, or UART interface. This allows for more flexible serial communication on a single microcontroller by using the predefined SERCOM pin mappings [1, 5]. To know which pins you need for which interfaces, one must consult the official documentation of the microcontroller and the datasheet of the chip used in the board [4, 1].

The main reason to prefer SERCOM modules in microcontrollers over hardwired UART or SPI blocks is to have more flexibility with connectivity on the board [1]. Older microcontrollers are limited by fixed configurations, where certain pins only act as a specific interface.

For PCB designers, this architecture makes it easier to allocate hardware resources based on project needs, such as needing multiple UARTs and no SPI interface. For example, a designer can configure the needed SERCOM in the chip to UART mode; once configured, that specific SERCOM works strictly as a UART interface [5].

2.2 Principles of UART

The easiest way to explain asynchronous communication is through its counterpart, synchronous communication, where parties share a physical clock line. In UART, there is no shared clock line, but the parties still need to be synchronized in time using predefined Baud rates [7]. To start data transfer, the transmitting UART pulls the idle high line to low for exactly one bit period, which is called the **start bit** [4, 6]. This acts as a timing anchor for the receiver.

A standard UART frame consists of this start bit, a **data frame** (5 to 9 bits), an

optional **parity bit**, and one or two **stop bits** that return the line back to a high voltage state. The parity bit serves as an error handling bit; if it is 0 (even parity), the total count of 1s in the data frame should be even, and if it is 1 (odd parity), the total should be odd [6]. Matching the Baud rates is necessary because the receiver starts reading the data according to the agreed rate immediately after detecting the start bit. If this rate is inconsistent, bits of the transmission might be missed or misinterpreted due to timing errors [7, 6].

2.3 Principles of SPI

"**SPI** (Serial Peripheral Interface) is a synchronous, full duplex main-subnode-based interface" [3].

Synchronous communication is as defined in the previous section. On the hardware level, the communicating devices share a clock line which they use to synchronize their communication. The **master/slave** (also known as main/subnode or controller/target) architecture is based on the principle that the master device defines when, which device, and at what speed a slave device communicates.

The master selects the specific slave device it wishes to interact with, allowing it to communicate with that device exclusively while other slave devices remain idle, waiting for a command.

MOSI (Master Output Slave Input) and **MISO** (Master Input Slave Output) are the dedicated data lines in an SPI bus. MOSI transmits data from the master to the slave, while MISO transmits data from the slave back to the master [3]. The protocol synchronizes these transmissions using the **Serial Clock** (SCK). The clock speed can be adjusted according to the system's requirements; while this requires knowledge of the specific device's design limits, it allows for optimizing the interface for the best available performance [8].

CS (Chip Select) or **SS** (Slave Select) is used by the master to activate the target slave device for communication. Typically, each slave connects through its own dedicated CS line [8].

3 Programming Assignments

3.1 Written report

I won't explain every single detail that was already covered in the assignment instructions, such as basic SERCOM enabling. This is not because they aren't important, but because they were simpler to implement. Full documentation of the steps is included as comments in the source code.

We can enable the APBA clock for the SERCOM1 as follows [4, p. 164]:

```
MCLK->APBAMASK.reg |= MCLK_APBAMASK_SERCOM1
```

More specifically, we are setting the 13th bit in the APBAMASK register [4, p. 173]. This is critical; without this, nothing later will work, as no clock signal will be passed to the SERCOM module.

3.1.1 Configuring the peripheral clock

Using the default GCLK0 was fairly simple, but for practice, I wanted to switch to a different clock generator and modify its frequency.

3.1.2 GCLK0

With the line below, we set the GCLK0 value (0x0) to the register at position 0.

```
GCLK->PCHCTRL[GCLK_SERCOM1_CORE].reg = GCLK_PCHCTRL_GEN_GCLK0  
                                         | GCLK_PCHCTRL_CHEN;
```

GCLK_PCHCTRL_CHEN then sets the 6th bit to 1, which enables the peripheral channel [4, p. 155]. After looking into it, since the generator bits are 0 by default, it already points to GCLK0. I verified this by commenting out the generator selection and only setting the enable bit.

3.1.3 GCLK5

Using a custom clock generator required more steps, as seen in the code below. I chose the FDPLL200M0 oscillator (source value 7 in bits [3:0]) [4, p. 152]. To modify

the frequency, I explicitly **unset** the `DIVSEL` bit (bit 12) and set the `DIV` value to 10. With this configuration, we are just dividing the frequency by 10. The `DIV` field uses 16 bits [4, p. 151]. This results in a 12MHz clock, assuming the `FDPLL200M0` oscillator runs at 120MHz.

```
GCLK->GENCTRL[5].bit.DIVSEL = 0;
GCLK->GENCTRL[5].bit.DIV = 0xA;
GCLK->GENCTRL[5].reg = GCLK_GENCTRL_SRC(7)
    | GCLK_GENCTRL_GENEN;
```

After this, we need to wait until the generator synchronization is finished before continuing [4, p. 144]:

```
while (GCLK->SYNCBUSY.bit.GENCTRL5);
```

Finally, I set `GCLK5` as the controller for `SERCOM1` by writing 5 to bits [3:0] of the `PCHCTRL` register and enabling it via the `CHEN` bit.

```
GCLK->PCHCTRL[GCLK_SERCOM1_CORE].reg = GCLK_PCHCTRL_GEN_GCLK5
    | GCLK_PCHCTRL_CHEN;
```

3.1.4 Pin multiplexing and USART details

The `PMUX` register assigns the pins to the peripheral. I used `PMUXE` for pin 16 (even) and `PMUXO` for pin 17 (odd):

```
PORT->Group[0].PMUX[muxIndex].bit.PMUXE = 0x2;
PORT->Group[0].PMUX[muxIndex].bit.PMUXO = 0x2;
```

According to the datasheet, peripheral function C (0x2) is required for `SERCOM1` [4, p. 32, p. 844].

One more important detail is `SERCOM1->USART.CTRLA.bit.MODE = 0x1;`. This tells the `SERCOM` to operate in Internal Asynchronous mode, which is the core of this UART assignment.

3.2 Baud calculation

The baud calculation was simple, since the equation 3.1 was provided in the `SAMD` datasheet [4, p. 853].

$$BAUD = 65536 \cdot \left(1 - 16 \cdot \frac{f_{BAUD}}{f_{ref}} \right) \quad (3.1)$$

where:

- 65536 is the 16-bit register resolution (2^{16}),
- f_{BAUD} is the desired baud rate (e.g., 9600),
- f_{ref} is the internal generic clock frequency.

In code, we are doing the calculation at compile time:

```
constexpr bool USE_GCLK5 = false;

constexpr float DIVISOR = 10.0f;
constexpr float BASE_FREQ = 120000000.0f;
constexpr float F_REF =
    USE_GCLK5 ? BASE_FREQ / DIVISOR : BASE_FREQ;

constexpr float F_BAUD = 9600.0f;
constexpr float REGISTER_SIZE = 65536.0f;

constexpr uint16_t BAUD_RATE =
    (uint16_t)(REGISTER_SIZE * (1.0f - 16.0f * (F_BAUD / F_REF)));
```

So, depending on whether we are using the GCLK0 or GCLK5 implementation, we obtain the correct baud rate. Calculating the baud rate for GCLK0 gives us:

$$65536 \cdot \left(1 - 16 \cdot \frac{9600}{120000000} \right) \approx 65452$$

4 Lecture and Assignment Feedback

- **Assignments:** The assignment questions were clear, but the specific tips considering the datasheet were a bit fuzzy at first. For example, "Start at the beginning of the peripheral's chapter" took me a while to find out which peripheral we were talking about, especially while I was trying to figure out where to find specific information within the document. However, after spending time navigating the datasheet, the tips became much clearer and I stumbled upon the sequential instructions inside the document as well. The questions definitely helped develop my embedded programming skills and encouraged further research. From a research point of view, it is perfect to have such a comprehensive document to cite and gather information from. Once you know your invariants, you can research and apply them accordingly. This is especially useful for tasks like developing a driver for an OS targeting SAMD51 chips specifically.
- **Time Spent:** I spent around 20-25 hours on this assignment. The amount of code written is small, but the learning gathered from it is huge. I spent about 5 hours of that time writing the programming assignment report; it was a great way to reflect on what I have learned, and during the writing process, I actually rewrote parts of the code as well. I transitioned to using one-line register assignments and setting bits only where necessary to maintain a clear separation of configurations.
- **Overall Learning:** For me, the biggest insight was learning how to actually read and navigate a massive datasheet. I now feel like I have the skills to "fine-tune" applications to my needs when I have to go to the hardware level. I also learned that with HW operations, the order and timing really matter. I found out the hard way that I needed to wait for certain flags to flip; at first, the application did not function, but when I tried to print out what was wrong, it suddenly started working. That was a clear sign that the next step depended on an operation that didn't have enough time to finish yet.
- **Other Comments/Opinions:** I found the suggested reading strategy/tips very

helpful. Although it was a bit confusing at first, it became clearer as I got more comfortable with the document. I enjoyed this assignment; it is always fun to go down to the hardware level to get a peek "under the hood."

I also learned the hard way when I first tried to use pins PA20 (D9) and PA21 (D8). While trying to set PA20 as the transmit pin, I discovered that the specific value 0x1 for the transmit pad (TXPO) was reserved for that configuration, forcing me to switch to pins PA16 (D13) and PA17 (D12). Fortunately, because I had documented the relevant datasheet pages directly into the source code, swapping the pins was quite simple. However, it still required updating several other parts of the configuration, such as the MCLK mask and PMUX indices.

References

- [1] ADAFRUIT INDUSTRIES. Using ATSAMd21 SERCOM for more SPI, I2C and Serial ports. <https://learn.adafruit.com/using-atsamd21-sercom-to-add-more-spi-i2c-serial-ports/overview>, 2016. Read: 27.3.2026.
- [2] ADAFRUIT INDUSTRIES. Adafruit Metro M4 Express featuring ATSAMd51. <https://learn.adafruit.com/adafruit-metro-m4-express-featuring-atsamd51/overview>, 2018. Read: 27.3.2026.
- [3] DHAKER, P. Introduction to SPI Interface. <https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html>. Read: 2.4.2026.
- [4] MICROCHIP TECHNOLOGY INC. SAM D5x/E5x Family Data Sheet. <https://onlinedocs.microchip.com/oxy/GUID-F5813793-E016-46F5-A9E2-718D8BCED496-en-US-15/index.html>. Read: 27.3.2026.
- [5] MICROCHIP TECHNOLOGY INC. SERCOM USART on Microchip Cortex Devices (SAM and PIC32C). <https://onlinedocs.microchip.com/oxy/GUID-5ACFA734-B54D-4EC3-BED3-29919448E97E-en-US-1/index.html>. Read: 27.3.2026.
- [6] MICROCONTROLLERS LAB. UART Communication Introduction, Working, Frame Format, Applications. <https://microcontrollerslab.com/uart-communication-working-applications/>. Read: 2.4.2026.
- [7] PEÑA, E., JA LEGASPI, M. G. UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter. <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>. Read: 2.4.2026.

- [8] YIC INTERNATIONAL CO., LIMITED. Exploring Serial Peripheral Interface (SPI) Communication. [https://www.yic-electronics.com/blog/Exploring-Serial-Perripheral-Interface\(SPI\).html](https://www.yic-electronics.com/blog/Exploring-Serial-Perripheral-Interface(SPI).html). Read: 2.4.2026.